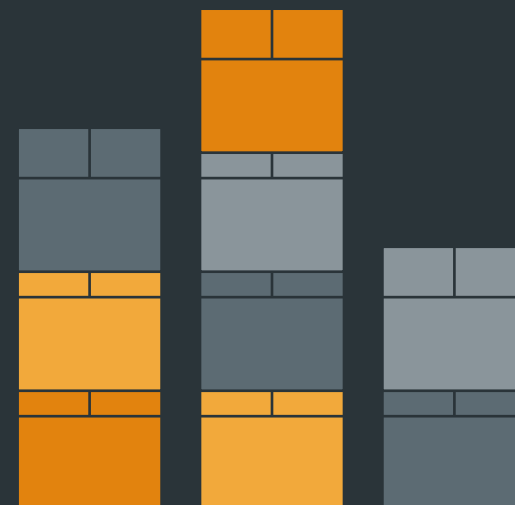


Warehouse Retrieval

Multi-Stack และ Greedy Selection

การออกแบบและวิเคราะห์อัลกอริทึมสำหรับหยิบกล่องเป้าหมายออกจากคลังสินค้าที่จัดเก็บแบบ Stack หลายกอง โดยเปรียบเทียบกลยุทธ์การเลือก Stack ปลายทางสองรูปแบบ

บทที่ 3 · การออกแบบอัลกอริทึม | Algorithm A: First Available Stack | Algorithm B: Best Fit Stack



บทนำและความสำคัญ

1

ที่มาและความสำคัญ

คลังสินค้าสมัยใหม่จัดเก็บกล่องสินค้าจำนวนมากในพื้นที่จำกัด จึงนิยมวางซ้อนกันเป็น Stack หลายกอง การหยิบกล่องที่ถูกฝังอยู่ด้านล่างจึงมีความซับซ้อนและต้องมีวิธีการที่มีประสิทธิภาพ

2

วัตถุประสงค์

ออกแบบและเปรียบเทียบอัลกอริทึมสำหรับย้ายกล่องที่กีดขวางไปยัง Stack อื่น เพื่อนำกล่องเป้าหมายออกมาได้อย่างถูกต้องและมีประสิทธิภาพ

3

ขอบเขตงาน

ศึกษาและเปรียบเทียบกลยุทธ์การเลือก Stack ปลายทาง 2 รูปแบบ คือ First Available Stack และ Best Fit Stack ภายใต้เงื่อนไขของปัญหาที่กำหนด

Case Story

คลังสินค้าวางกล่องซ้อนกันเป็นหลายกอง กองที่อยู่ด้านบนสามารถนำออกได้ทันที แต่หากกล่องเป้าหมายอยู่ด้านล่าง ต้องย้ายกล่องที่ขวางอยู่ไปยัง Stack อื่นก่อนจึงจะหยิบกล่องเป้าหมายออกมาได้

ขอบเขตงานหลัก

- 1 ย้ายได้เฉพาะกล่องด้านบนสุดของแต่ละ Stack
- 2 แต่ละ Stack มีความจุจำกัด (Capacity)
- 3 Stack ปลายทางต้องมีพื้นที่ว่างเพียงพอ
- 4 หลังนำกล่องเป้าหมายออก ต้องสามารถนำกล่องที่ย้ายกลับคืนได้
- 5 ไม่บังคับเงื่อนไขน้ำหนักกล่อง

ข้อมูลของกล่องแต่ละใบ

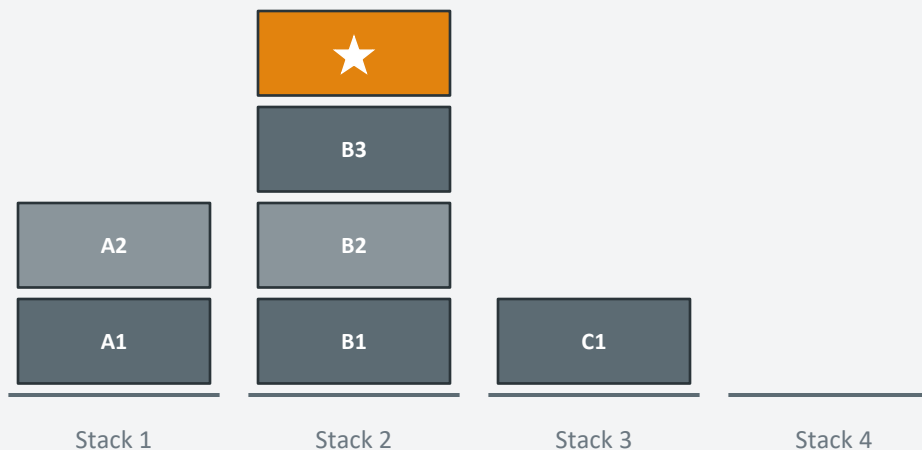
Box ID

Category

Arrival Time

Priority

ภาพสถานะ Stack (ตัวอย่าง)



★ = กล่องเป้าหมาย ต้องย้าย B3 ออกก่อนจึงจะหยิบได้

Input, Output และ Constraints

Input

- จำนวน Stack (N) และความจุต่อ Stack (C)
- สถานะกล่องเริ่มต้นในแต่ละ Stack
- ข้อมูลกล่อง: Box ID, Category,
- Arrival Time, Priority
- Box ID ของกล่องเป้าหมายที่ต้องหยิบออก

Output

- ลำดับการย้ายกล่อง (Move Sequence)
- จำนวนครั้งที่ย้ายกล่องทั้งหมด
- ตำแหน่งสุดท้ายของกล่องเป้าหมาย
- สถานะผลลัพธ์: สำเร็จ / ไม่สำเร็จ
- สถานะ Stack ทั้งหมดหลังดำเนินการ

Constraints

- ย้ายได้เฉพาะกล่องบนสุดของ Stack
- แต่ละ Stack มีความจุจำกัด
- Stack ปลายทางต้องมีพื้นที่ว่าง
- ต้องนำกล่องที่ย้ายกลับคืนได้
- ไม่มีเงื่อนไขเรื่องน้ำหนักกล่อง

Algorithm A — First Available Stack

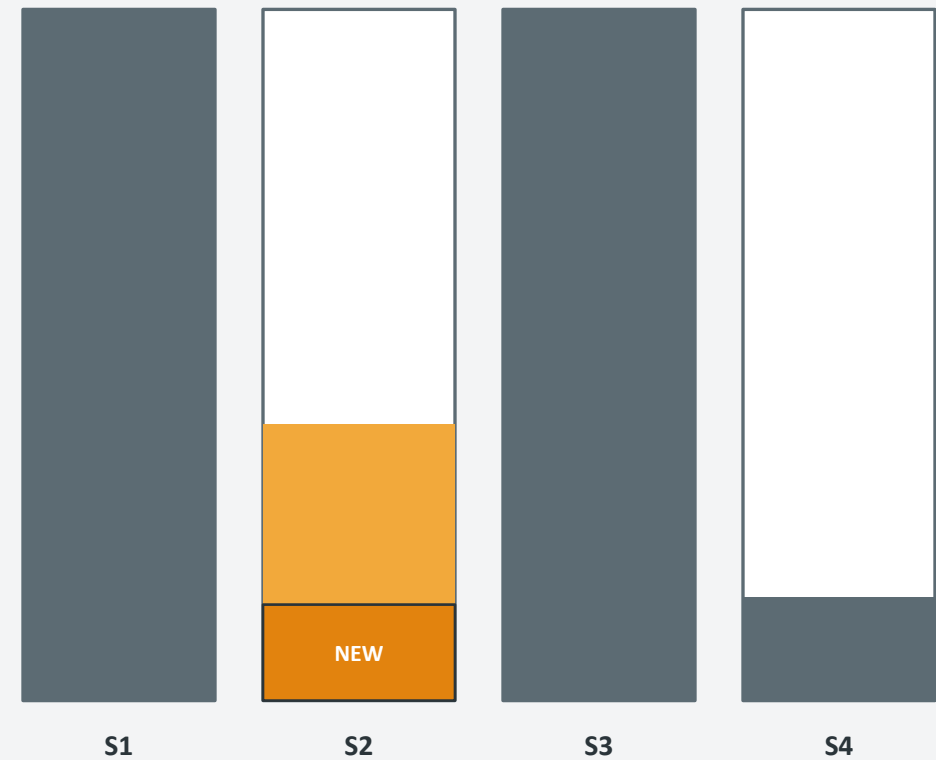
เลือก Stack ปลายทางเป็น Stack แรกที่พบว่ามีพื้นที่ว่าง โดยไล่ตรวจสอบตามลำดับดัชนี (index) โดยไม่คำนึงว่า Stack นั้นจะเหลือพื้นที่มากหรือน้อยเพียงใด — เน้นความเร็วในการตัดสินใจ

```
function SelectDestination_A(stacks, source):  
    for i = 0 to N-1:  
        if i != source AND  
           stacks[i].size < capacity:  
            return i      // Stack แรกที่ว่าง  
    return NOT_FOUND
```

✓ ข้อดี: ค้นหาโดยเฉลี่ยตรวจสอบเพียง $N/2$ ครั้ง

✗ ข้อเสีย: พื้นที่กระจายไม่สม่ำเสมอ อาจเหลือ Stack ว่างไม่คุ้มค่า

ตัวอย่าง: เลือก Stack ปลายทาง



เต็ม

ตรวจสอบ #1 → ว่าง
เลือกเลย

เต็ม

ไม่ถูกตรวจสอบ

Algorithm B — Best Fit Stack

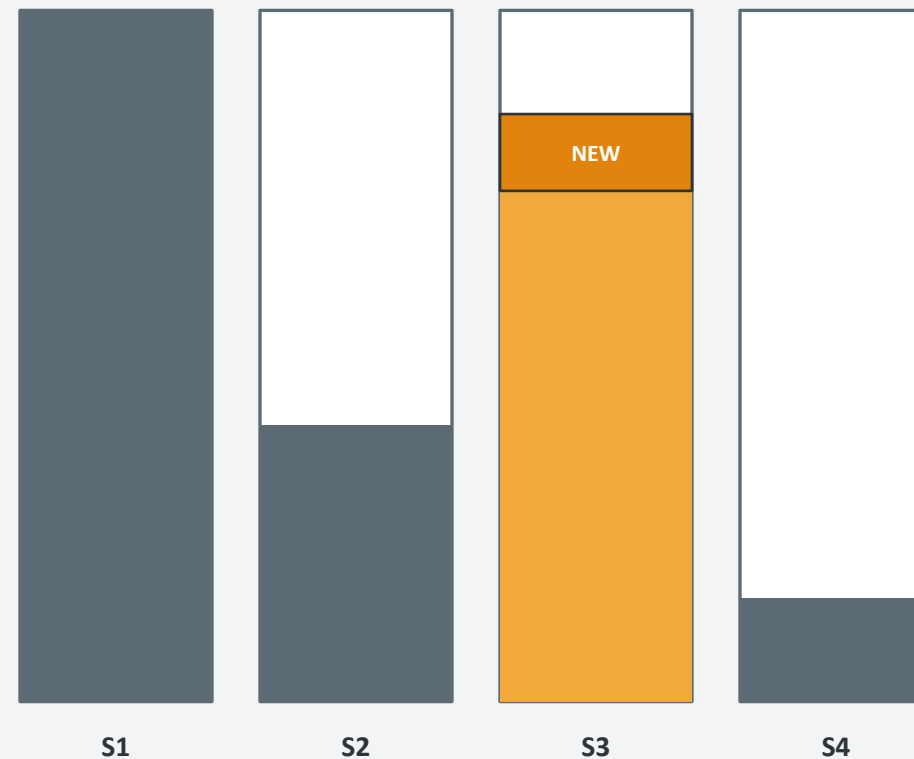
ตรวจสอบ Stack ที่ยังไม่เต็มทั้งหมด แล้วเลือก Stack ที่เหลือพื้นที่ว่างน้อยที่สุดหลังจากวางกล่องลงไป (fit แน่นที่สุด) — เน้นการใช้พื้นที่คลังสินค้าอย่างคุ้มค่า

```
function SelectDestination_B(stacks, source):  
    best = NOT_FOUND; bestRemain = ∞  
    for i = 0 to N-1:  
        if i != source AND stacks[i].size < capacity:  
            remain = capacity - stacks[i].size  
            if remain < bestRemain:  
                bestRemain = remain; best = i  
    return best
```

✓ ข้อดี: ใช้พื้นที่คุ้มค่า ลดโอกาส Stack เต็มจนหาที่ว่างไม่ได้

✗ ข้อเสีย: ต้องตรวจสอบทุก Stack เสมอ จึงช้ากว่าในค่าเฉลี่ย

ตัวอย่าง: เลือก Stack ปลายทาง



เต็ม

เหลือว่างมาก

เหลือว่างน้อยสุด → เลือก

เหลือว่างมาก

วิธีทำแบบ Step-by-step

1

ค้นหา **Stack** ต้นทาง

ค้นหาว่ากล่องเป้าหมาย (Target Box ID) อยู่ใน Stack ไດ



2

ตรวจสอบตำแหน่งบนสุด

หากกล่องเป้าหมายอยู่บนสุดอยู่แล้ว ให้หยิบออกได้ทันที (จบการทำงาน)



3

เลือก **Stack** ปลายทาง

ใช้ Algorithm A หรือ B เพื่อเลือก Stack ที่จะย้ายกล่องบนสุดไปพักไว้

4

ย้ายกล่องที่เกิดขวาง

ย้ายกล่องบนสุดไปยัง Stack ปลายทาง แล้วบันทึกการย้าย (Move Log)



5

ทำซ้ำจนถึงเป้าหมาย

วนขั้นตอน 2-4 จนกล่องเป้าหมายอยู่บนสุด แล้วหยิบออก



6

คืนกล่องกลับ (**Restore**)

นำกล่องที่ถูกย้ายพักไว้ กลับคืน Stack เดิมตามลำดับย้อนกลับของ Move Log

การอธิบายความถูกต้อง (Correctness Argument)

1 Precondition

กล่องเป้าหมายต้องมืออยู่จริงในคลัง และผลรวมพื้นที่ว่างของ Stack อื่นต้องเพียงพอสำหรับกล่องที่ต้องย้ายทั้งหมด

2 Invariant

ตลอดการทำงาน กล่องเป้าหมายจะยังคงอยู่ใน Stack ต้นทางเสมอ และจะไม่ถูกย้ายออกจนกว่าจะขึ้นมาอยู่บนสุด

3 Postcondition

กล่องเป้าหมายถูกนำออกสำเร็จ กล่องที่ถูกย้ายทุกใบอยู่ใน Stack ที่ถูกต้องตามความจุ และ Move Log สมบูรณ์พร้อมย้อนกลับได้

4 Termination

จำนวนกล่องที่กีดขวาง (depth) ลดลง 1 ทุกรอบการวนซ้ำ และมีค่าจำกัด จึง loop สิ้นสุดเสมอภายใน d รอบ

Big-O Analysis

N = จำนวน Stack, C = ความจุต่อ Stack, d = ความลึกของกล่องเป้าหมาย (จำนวนกล่องที่กีดขวาง)

กรณี	จำนวนการย้าย	Algorithm A (ค้นหาปลายทาง)	Algorithm B (ค้นหาปลายทาง)
Best Case	0 (อยู่บนสุด)	$O(1)$	$O(1)$
Average Case	d ครั้ง	$O(d \cdot N/2)$	$O(d \cdot N)$
Worst Case	$d = C - 1$ ครั้ง	$O(d \cdot N)$	$O(d \cdot N)$

$O(d \cdot N)$

Time Complexity

worst case ทั้งสองอัลกอริทึม

$O(N \cdot C)$

Space Complexity

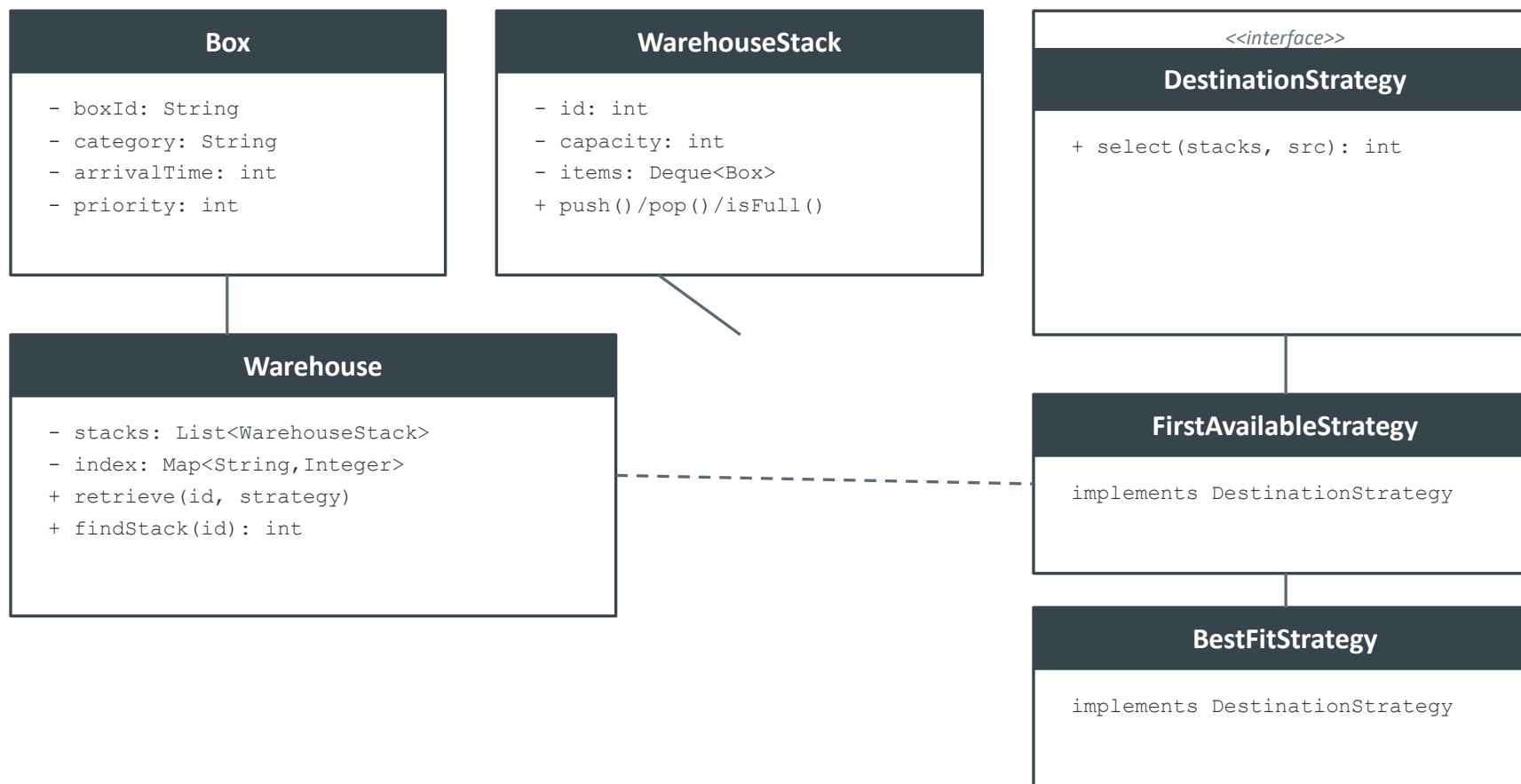
พื้นที่เก็บสถานะกล่องทุก Stack

$O(1)$

การค้นหา Stack ต้นทาง

เมื่อใช้ HashMap เก็บตำแหน่งกล่อง

Class Diagram และ Java Implementation



```

Box retrieve(String id,
             Strategy s) {
    int src=findStack(id);
    if (src==-1)
        throw new NotFound();
    while (!top(src)
           .equals(id)) {
        int dst=s.select(
            stacks, src);
        if (dst==-1)
            throw new NoSpace();
        Box b = pop(src);
        push(dst, b);
        log.add(new Move(
            b, src, dst));
    }
    return pop(src);
}
  
```

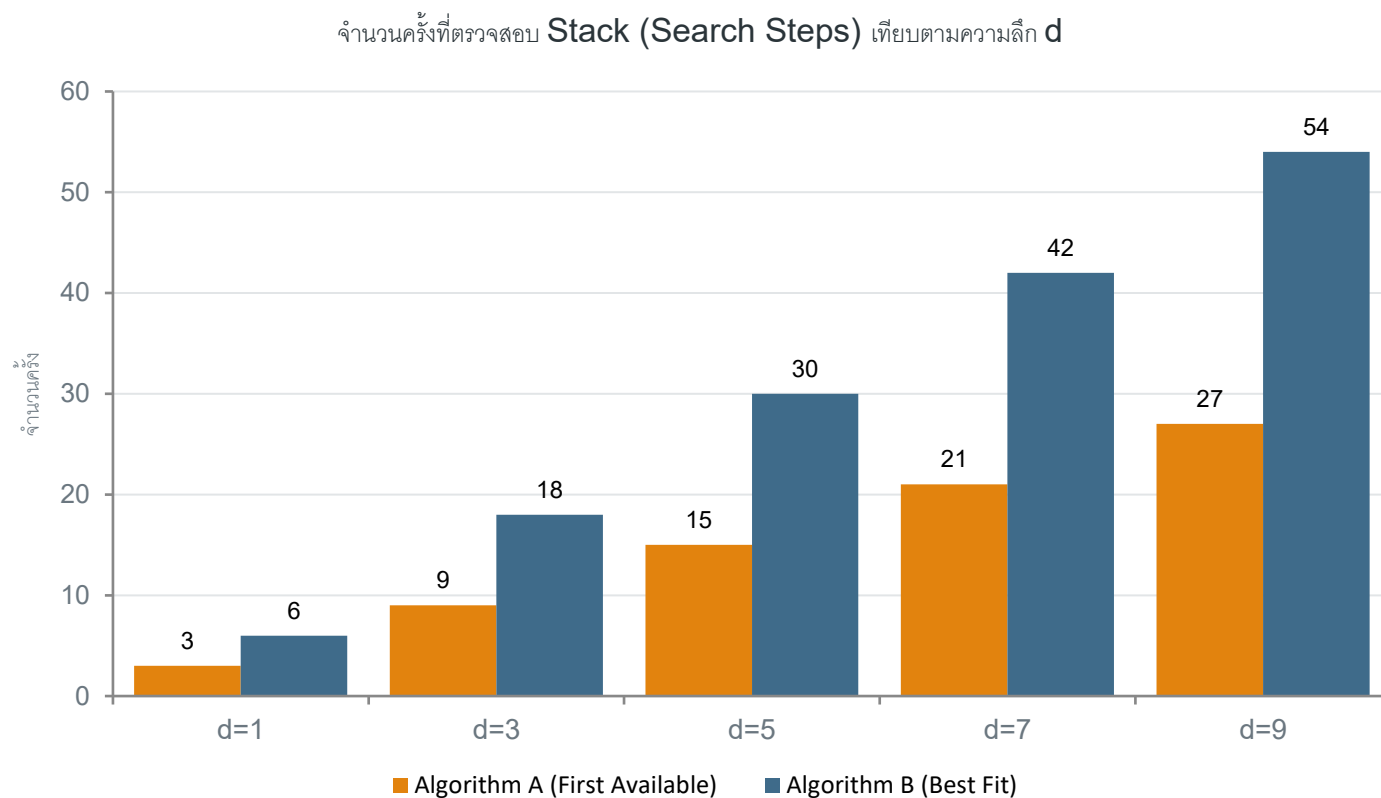
เส้นประ = dependency, เส้นทึบ = association/composition

Test Cases บังคับ

ID	เงื่อนไขทดสอบ	พฤติกรรมที่คาดหวัง
TC01	กล่องเป้าหมายอยู่ด้านบน	หยิบออกได้ทันที (0 การย้าย)
TC02	กล่องเป้าหมายอยู่ตรงกลาง	ย้ายกล่องด้านบนออกก่อน แล้วจึงหยิบได้
TC03	กล่องเป้าหมายอยู่ด้านล่างสุด	ย้ายกล่องทั้งหมดด้านบนออกก่อน (จำนวนย้ายมากที่สุด)
TC04	มี Stack วางอยู่ในระบบ	Stack วางถูกพิจารณาเป็นปลายทางที่ใช้ได้
TC05	Stack ปลายทางบางกองเต็ม	อัลกอริทึมข้าม Stack ที่เต็มไปยังกองถัดไป
TC06	ไม่มีพื้นที่เพียงพอในระบบ	คืนค่าล้มเหลว (Fail) พร้อมเหตุผลที่ชัดเจน
TC07	ไม่พบกล่องเป้าหมายในคลัง	คืนค่า Not Found โดยไม่ทำการย้ายใดๆ
TC08	มี Stack ปลายทางให้เลือกหลายกอง	เปรียบเทียบผลลัพธ์ระหว่าง Algorithm A และ B

Experimental Results

ตัวอย่างผลจำลอง (Simulation) — $N = 6$ Stack, Capacity = 10 ต่อ Stack, วัดจำนวนครั้งที่ตรวจสอบ Stack เพื่อเลือกปลายทาง



ข้อสังเกตจากผลจำลอง

- จำนวนการ ย้ายกล่อง เท่ากันทั้งสองอัลกอริทึม ($= d$) เพราะขึ้นกับความลึกของกล่องเป้าหมายเท่านั้น
- Algorithm A ใช้จำนวนครั้งที่ตรวจสอบน้อยกว่าโดยเฉลี่ย เพราะหยุดค้นหาทันทีที่พบ Stack ว่างแรก
- Algorithm B ตรวจสอบทุก Stack เสมอ จึงใช้เวลาค้นหามากกว่า แต่ได้การใช้พื้นที่ที่คุ้มค่ากว่า
- ยิ่ง d มาก ความต่างของจำนวนครั้งที่ตรวจสอบยิ่งชัดเจนขึ้นตามสัดส่วนเชิงเส้น

สรุปและอภิปรายผล

วิธีที่เหมาะสมกว่า / เหตุผลในการเลือก

- หากเน้นความเร็ว และพื้นที่คลังยังไม่ตึงตัว → เลือก Algorithm A
- หากเน้นการใช้พื้นที่คุ้มค่า และใกล้เคียงความจุ → เลือก Algorithm B

ข้อจำกัดของงาน

- พิจารณาการหยิบที่ละกล่อง ยังไม่รองรับหลายคำขอพร้อมกัน
- ยังไม่นำเงื่อนไขน้ำหนักหรือลำดับความสำคัญมาใช้เลือก Stack ปลายทาง

แนวทางพัฒนาต่อ

- ผสานกลยุทธ์แบบ Hybrid ที่ปรับตามระดับความเต็มของคลัง
- เพิ่มการจัดลำดับตาม Priority และรองรับการหยิบหลายกล่องพร้อมกัน

สิ่งที่ได้เรียนรู้

- การแลกเปลี่ยนระหว่างความเร็วในการค้นหา กับประสิทธิภาพการใช้พื้นที่
- การพิสูจน์ความถูกต้องด้วย Invariant และการวิเคราะห์ Big-O อย่างเป็นระบบ